

PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC-2685 ROBÓTICA MÓVIL

LAB 2: CONTROL EN LAZO CERRADO Y COMPORTAMIENTOS REACTIVOS

Fecha de Presentación: miércoles 28 de octubre de 2026

Introducción

Vamos avanzando en el control de los movimientos de los TurtleBots. En el Lab 1 aprendieron a mover el robot y a definir trayectorias (lista de poses) como una sucesión de distancias que fueron calculadas a partir de una velocidad predefinida y tiempos de aplicación (control en lazo abierto). Además, lograron programar una rutina de percepción básica para la de detección de obstáculos.

En esta oportunidad daremos el siguiente paso en el control de movimiento del robot, y lo dotaremos con la capacidad de cuantificar su desplazamiento a través de la odometría. En esta experiencia supondremos que la odometría es "perfecta", es decir, que no existe error entre la posición real del robot y la información entregada por el sensor. Para este laboratorio, programarán un conjunto de rutinas para las cuales el control realimentado es fundamental. Dichas rutinas consisten en: llevar al robot a una posición deseada minimizando el error, navegar por un pasillo complejo sin colisionar con las paredes y seguir una trayectoria predefinida.

1. Navegación utilizando control P y PI (Simulador)

Como vimos en el laboratorio anterior, si implementamos el sistema de navegación de nuestro robot a partir de un controlador en lazo abierto (sin realimentación de la odometría), los movimientos obtenidos serán imprecisos (sobre todo los giros). Para mejorar la calidad de nuestro sistema de navegación, en esta experiencia usted deberá implementar un lazo cerrado de control como el que se muestra en la figura 1.

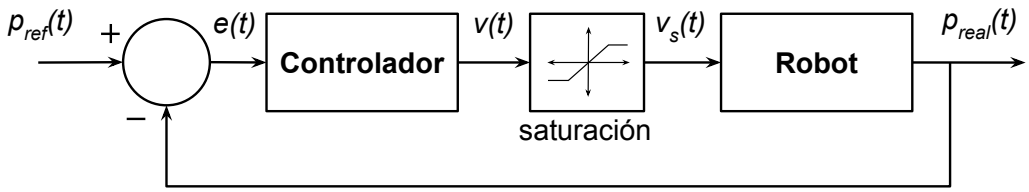


Figura 1: Diagrama de control en lazo cerrado con saturación en actuación.

Para su implementación, usted deberá crear un nodo que posea la misma jerarquía de funciones (o métodos) que implementó para el nodo `dead_reckoning_nav` del laboratorio 1, pero aplicando las siguientes modificaciones:

Nivel 3: `accion_mover_cb( msg )` No requiere cambios.

Nivel 2: `mover_robot_a_destino( goal_pose )` Esta función recibirá una pose en formato (x, y, θ), la cual deberá ser descompuesta en una lista de desplazamientos lineales o rotacionales (solo uno a la vez)

en formato $(linear_disp, rotational_disp)$. Por ejemplo, si esta función recibe como argumento la pose $(1, 1, \pi/2)$, su función deberá generar la siguiente lista de desplazamientos: $[(1, 0), (0, \pi/2), (1, 0)]$.

Nivel 1: `aplicar_velocidad( displacement_list )` Esta función recibirá como entrada la lista de desplazamientos calculados por la función `mover_robot_a_destino`. Cada uno de los elementos de la lista será utilizado como señal de referencia para un controlador de desplazamiento lineal (ej: $(1, 0)$), o bien, para un controlador de desplazamiento angular (ej: $(0, \pi/2)$). Como consecuencia de lo anterior, usted deberá implementar dos controladores de la familia PID para controlar los movimientos de su robot.

En este laboratorio, usted deberá controlar las variables *desplazamiento lineal* y *desplazamiento angular* utilizando dos contralodores independientes (uno para movimiento lineal, y otro para movimiento rotacional). Para poder facilitar la implementación de los controladores, usted deberá basarse en la clase *PController* publicada en el portal del curso. En dicha clase, ya se encuentra imlementada la lógica de un controlador P, y usted deberá agregar el código necesario que permita incorporar la parte integrativa I. Una vez finalizada la imlementación del controlador, usted deberá experimentar los dos variantes: Controlador P y Controlador PI. Para dichos experimentos, usted deberá ajustar los parámetros del controlador hasta lograr que el robot se mueva con la mayor precisión posible. En resumen, usted deberá ajustar los parámetros de 4 controladores según se detalla a continuación:

- Caso control P: 1 controlador P para desplzamiento lineal y 1 controlador P para desplazamiento angular
- Caso control PI: 1 controlador PI para desplzamiento lineal y 1 controlador PI para desplazamiento angular

Para determinar el valor de los parámetros K_p y K_i, deberá seguir el método de *prueba y error* hasta alcanzar el menor error posible en la trayectoria descrita por el robot. Se recomienda comenzar a probar con valores bajos para ambos parámetros (del orden de 0.01), y posteriormente aumentarlos gradualmente hasta alcanzar un movimiento satisfactorio. Establezca una velocidad máxima de desplazamiento de 0.2 [m/s] a través de una función de saturación como la esquematizada en la figura 1. Para ello, modifique su controlador de manera tal que permita limitar la actuación máxima dada por el controlador.

Una vez finalizadas las etapa de implementación y de establecimiento de parámetros, deberá repetir la rutina *Avanzar y Rotar* solicitada en el Laboratorio 1. Estas deberán ser evaluadas tanto para el controlador P como para el controlador PI.

Incluir en presentación:

- Gráfico con señales de: referencia $P_{ref}(t)$, actuación $V_s(t)$ y salida $P_{real}(t)$, para ambos controladores (P y PI) y ambas variables de control (lineal y angular). En total, debe presentar 4 gráficos. Sea cuidadoso con la escala escogida para la variable independiente ya que en su gráfico debe notarse la evolución de cada variable.
- Gráfico con trayectoria seguida por el robot en el plano bidimensional para controlados P y PI, comparándolo con el gráfico de trayectoria obtenido por el algoritmo *dead reckoning* implementado en el laboratorio 1.

Incluir en paquete: archivo launch de nombre *avanzar_y_rotar_ctrl.xml* que permita iniciar tanto los nodos que implementan su rutina, como los nodos que implementan el simulador. Deben garantizar que sea posible correr el experimento completo simplemente ejecutando:

```
ros2 launch <nombre_paquete> avanzar_y_rotar_ctrl.xml
```

Demostración: Movimiento en cuadrado controlado.

2. Seguimiento de trayectorias (Simulador)

Una de las tareas más importantes en robótica móvil es el planeamiento de rutas. Esta es una tarea que consiste en un proceso deliberativo, mediante el cual, un robot puede definir una trayectoria dentro de un mapa. Mediante dicha trayectoria, el robot podrá alcanzar una posición objetivo sin colisionar con obstáculos fijos, como por ejemplo, paredes, muebles, etc. Una vez que la trayectoria fue planificada, ésta es obtenida como una secuencia de coordenadas (en nuestro caso 2D: (x,y)) que guiarán al robot hasta su posición de destino.

Tal como vimos en clases, para poder ejecutar una trayectoria contamos con el método *Follow The Carrot*. Este métod utiliza un controlador de la familia PID para poder seguir un punto distante (*carrot*), que se va moviendo sobre la trayectoria a una distancia fija del punto más cercano al robot.

En esta sección, usted deberá implementar el método *Follow The Carrot* utilizando un controlador de tipo P, PI o PID, hasta encontrar el que permite seguir la trayectoria de mejor forma posible. Para esta experiencia, usted contará con tres archivos de texto que contienen las coordenadas que definen las trayectorias de la figura 2. Estas coordenadas representarán los puntos (x, y) del espacio en el simulador, en el sistema métrico.

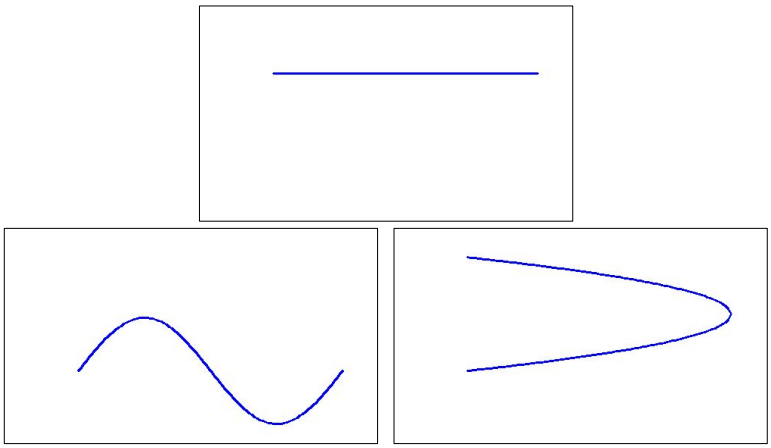


Figura 2: Trayectorias planeadas para seguimiento.

Para la implementación del método *Follow The Carrot*, deberá diseñar un controlador de la familia PID que controlará la orientación del robot, mientras aplica una velocidad lineal fija de 0.1 [m/s]. Este controlador recibirá como entrada: la orientación actual del robot (salida de la planta: $y(t)$) y el ángulo entre el centro del robot y la *zanahoria* (señal de referencia: $r(t)$). En base a estas entradas, el controlador enviará una velocidad rotacional a modo de actuación ($u(t)$), que intentará llevar a cero la diferencia entre los ángulos enviados.

Para interactuar con el controlador, implemente el nodo `follow_the_carrot` que deberá calcular el ángulo actual del robot y el ángulo formado entre el robot y la *zanahoria*, para luego enviarlos al controlador. Para el primer ángulo, use la información proveniente de la odomatría del robot. Para el segundo ángulo, usted primero deberá enviar la trayectoria hacia el nodo `follow_the_carrot` a través del tópico `nav_plan`, utilizando un mensaje de tipo `nav_msgs/Path` [1]. Para ello, implemente un cliente que lea cada archivo de texto con las coordenadas que definen la trayectoria, y encapsúlelo en el mensaje `nav_msgs/Path` para su envío. Una vez que el nodo `follow_the_carrot` cuente con la trayectoria, calcule el ángulo que forma el centro del robot con el punto de la trayectoria donde se encuentra la *zanahoria*, y envíelo como entrada al controlador.

Incluir en presentación: Gráfico con la trayectoria lograda versus la trayectoria original. Calcule el error cuadrático entre las curvas para las 3 trayectorias, y repórtelo en sus slides.

Incluir en paquete: archivo launch de nombre *follow_the_carrot.xml* que permita iniciar tanto los nodos que implementan su rutina, como los nodos que implementan el simulador. Deben garantizar que sea posible correr el experimento completo simplemente ejecutando:

```
ros2 launch <nombre_paquete> follow_the_carrot.xml
```

Demostración: Seguimiento de ruta para las tres secuencias entregadas. Durante la ejecución de las trayectorias, deberá levantar una ventana adicional donde se vea la trayectoria a seguir, y se aprecie la trayectoria real seguida por el robot en la medida que va avanzando.

3. Comportamientos reactivos usando control P (TurtleBot real)

Hasta este momento, nuestros TurtleBots solo han percibido el mundo que los rodea a través de la odometría y cámara de profundidad. Ahora, ha llegodo el momento de mejorar su conocimiento del mundo a través de la información de alto nivel que nos entrega la cámara RGB. En particular, en esta sección usted deberá implementar una rutina para detectar un cuadrado de color azul dentro de la imagen percibiada por tu robot, para luego poder seguirlo de forma controlada.

Para obtener la imagen RGB, usted deberá subscribirse al siguiente tópico utilizando el tipo de dato `Image`:

- `/camera/rgb/image_color`

Según lo anterior, en esta sección debe programar los siguientes dos nodos:

`detector_de_objeto` Como estamos dando los primeros pasos, por ahora solo utilizaremos un cuadrado de color azul sobre un fondo blanco (en la práctica, un hoja tamaño carta impresa con un cuadrado azul de 14×14 [cm]). Usando la imagen RGB, el nodo debe publicar la posición horizontal x del cuadrado azul con respecto al centro de la imagen en un tópico llamado `blue_square_position` (usted deberá escoger el tipo de mensaje más apropiado según el problema). Para hacer su detección más robusta, se recomienda que tranforme su imagen desde el espacio de color RGB al espacio HSV (para realizar esto con OpenCV, visite [2]). Recuerde que podría ser útil usar la imagen de profundidad para discriminar entre objetos de un mismo color según su distancia.

`seguidor_de_objeto` Este nodo estará encargado de leer la posición x del cuadrado azul desde el tópico `blue_square_position`, para luego hacer rotar al robot hasta que dicha figura se vea en el centro de la imagen RGB. Para ello deberá utilizar un controlador P usando la clase PController utilizada en la actividad anterior, establecer la referencia a un valor constante igual a cero, y definir como variable de control $y(t)$ la posición en la cual fue detectado el cuadrado. Una vez que logrado el seguimiento del cuadrado por medio de rotaciones, aplique una velocidad lineal constante de 0.2 [m/s] para hacer avanzar al robot mientras usted lo guía por el espacio con el cuadrado.

Incluir en paquete: archivo launch de nombre *seguimiento_de_objeto.launch* que permita iniciar tanto los nodos que implementan su rutina, como los nodos requeridos por el turtlebot. Deben garantizar que sea posible correr el experimento completo simplemente ejecutando:

```
ros2 launch <nombre_paquete> seguimiento_de_objeto.launch
```

Demostración: Deberá demostrar que puede guiar a tu robot por el espacio simplemente mostrandole el cuadrado azul.

Referencias

[1] https://docs.ros.org/en/humble/p/nav_msgs/msg/Path.html

[2] https://docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html